

# MyHeath

Plateforme de telemedecine & sante connectee

## Rapport de Projet de Fin d'Etudes

Backend ASP.NET Core 8 | Frontend React | MongoDB | Docker

Annee universitaire 2025-2026

Stack cible: .NET (C#) + React + MongoDB Atlas / Docker

# Resume / Abstract

## Resume (FR)

MyHeath est une plateforme de telemedecine orientee sante feminine et masculine, avec suivi quotidien, diabete, cycle menstruel, messagerie chiffree, coach IA (Claude) et console d'administration (utilisateurs, abonnements). Ce rapport presente la version academique dont le backend est realise en ASP.NET Core 8 (C#), en conservant React (Vite) et MongoDB. L'accent est mis sur l'architecture en couches, la securite (JWT, BCrypt, AES-256-CBC), le deploiement Docker et les scenarios cloud (Vercel pour le frontend, conteneurs pour l'API .NET).

## Abstract (EN)

MyHeath is a telemedicine platform for women's and men's health tracking, diabetes care, encrypted messaging, AI coaching, and admin subscriptions. This report documents the ASP.NET Core 8 backend rewrite while keeping React and MongoDB, including Docker deployment and cloud hosting considerations.

# 1. Introduction et contexte

## 1.1 Problematique

L'accès à un suivi de santé continu (cycle, glycémie, bien-être) reste fragmenté. MyHeath centralise le parcours patient-médecin avec une API sécurisée et une UI moderne.

## 1.2 Objectifs du projet

- Construire une API REST professionnelle en ASP.NET Core 8
- Conserver le frontend React existant (contrat /api compatible)
- Persister les données dans MongoDB (document-oriented)
- Chiffrer les données sensibles (AES-256-CBC) et authentifier via JWT
- Ajouter des fonctions avancées: RDV, rappels médicaments, notifications, audit
- Documenter Docker et le déploiement

## 1.3 Pourquoi .NET cette année

Le choix d'ASP.NET Core s'aligne sur les compétences acquises en formation (C#, Web API, middleware, DI, JWT). .NET 8 offre performances, typage fort, Swagger natif et un excellent support Docker (images [mcr.microsoft.com/dotnet](https://mcr.microsoft.com/dotnet)).

## 2. Analyse des besoins

### 2.1 Acteurs

- Patient (femme / homme): suivi, diabete, IA, RDV, médicaments
- Medecin: consultations, RDV, messagerie
- Administrateur: utilisateurs, plans, abonnements, audit

### 2.2 Exigences fonctionnelles

- Auth (register/login), RBAC patient|doctor|admin
- Suivi quotidien + score predictif 0-100
- Gestion periodes / insights cycle
- Dossier de sante chiffre
- Chat chiffre patient-medecin
- AI Coach (Anthropic Claude)
- Abonnements FREE / CARE / PREMIUM
- Rendez-vous telemedecine
- Rappels de médicaments + notifications

### 2.3 Exigences non fonctionnelles

- Securite: JWT 7j, BCrypt cost 12, AES au repos
- Portabilite: Docker Compose
- Maintenabilite: controllers / services / models
- Observabilite: healthcheck + Swagger

Figure 2.3 — Use Case Diagram

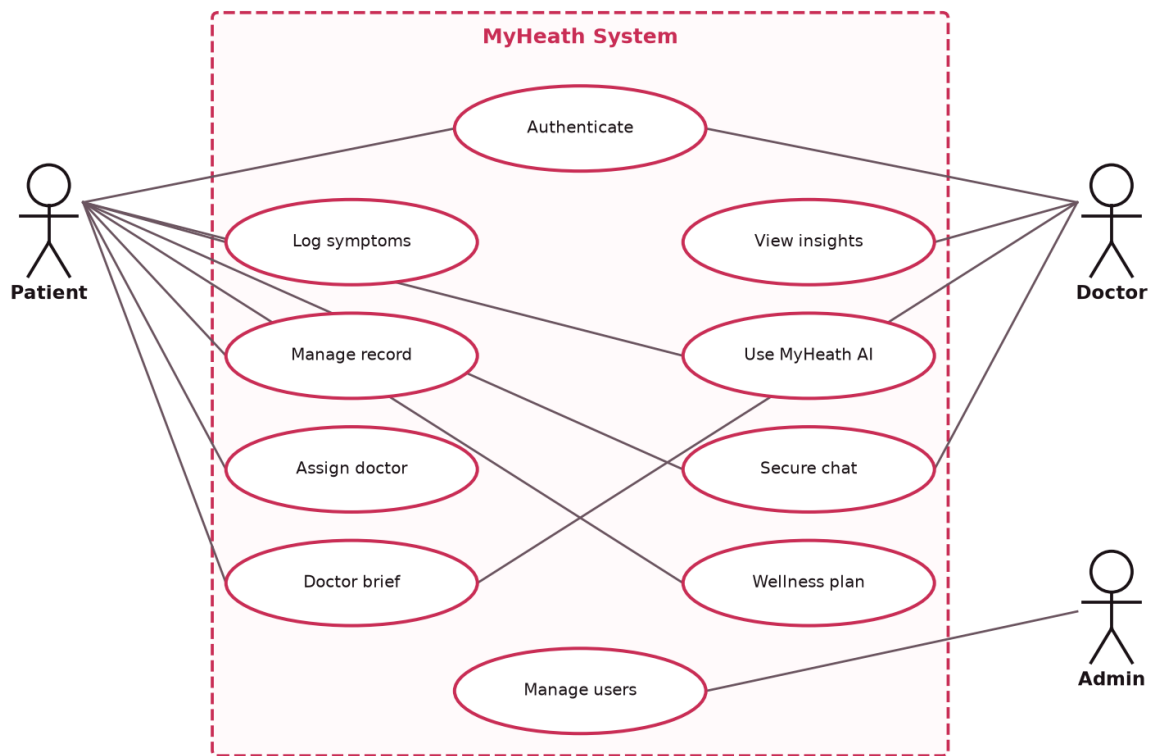


Figure: diagramme de cas d'utilisation (MyHeath)

### 3. Architecture technique

#### 3.1 Vue d'ensemble

Architecture 3-tiers: React (presentation) -> ASP.NET Core Web API (metier) -> MongoDB (persistance). Le frontend appelle VITE\_API\_URL (ex: http://localhost:5080/api).

**Figure 2.1 — MyHeath 3-tier MERN Architecture**

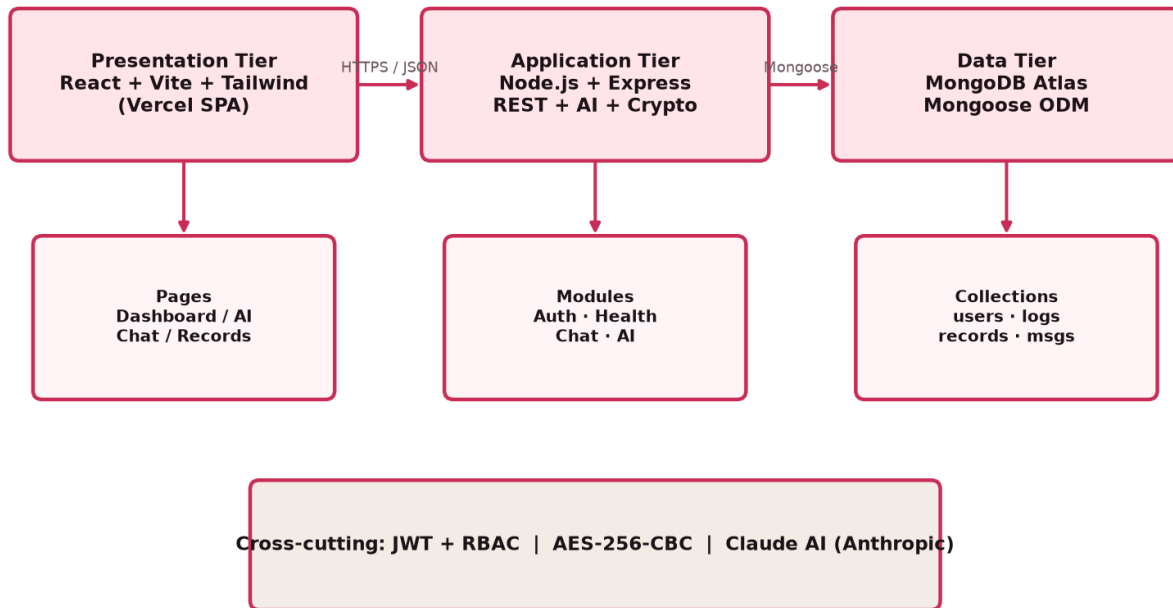


Figure: architecture logique 3-tiers

#### 3.2 Stack

- Backend: ASP.NET Core 8, C#, MongoDB.Driver, JWT Bearer, BCrypt.Net-Next, Swashbuckle
- Frontend: React 18, Vite, Tailwind, Recharts, Axios
- Base: MongoDB 7 (local Docker) ou MongoDB Atlas
- IA: Anthropic Messages API (Claude)

#### 3.3 Structure du projet .NET

```

backend-dotnet/
  MyHeath.Api/
    Controllers/  Auth, Health, Suivi, Chat, Ai, Admin,
                  Appointments, Notifications, Medications
    Models/       Entities (User, DailyHealthLog, ...)
    Services/     Mongo, AES, JWT, Claude, Seed, HealthScore
    Program.cs    DI, CORS, Swagger, Auth
    Dockerfile
  docker-compose.dotnet.yml
  
```

#### 3.4 Compatibilite avec l'ancien backend Node

Les routes REST restent sous /api/\* avec les memes payloads JSON camelCase. Le JWT porte la claim id; AES

utilise `iv_hex:ciphertext_hex` (SHA-256 du secret). Ainsi le meme frontend React peut basculer de Node (port 5000) vers .NET (port 5080).

## 4. Conception detaillee

### 4.1 Modele de donnees MongoDB

- users, healthrecords, symptomlogs, dailyhealthlogs, messages
- subscriptionplans, usersubscriptions
- appointments, appnotifications, medicationreminders, auditlogs

**Figure 2.6 — Domain Class Diagram**

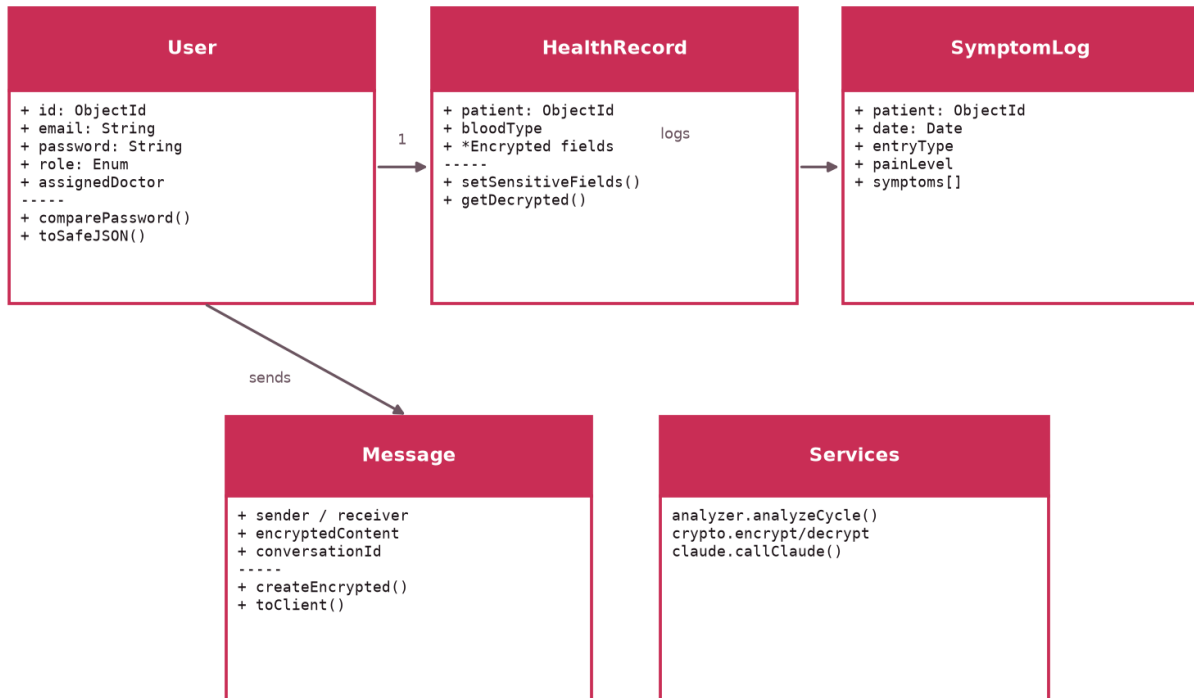


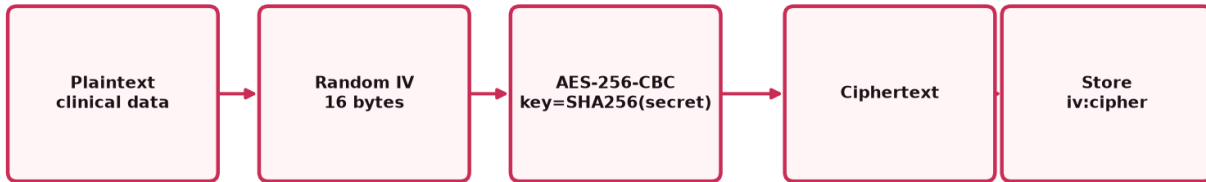
Figure: modele de classes / domaines

### 4.2 Securite AES-256-CBC

Les champs sensibles du dossier medical et le contenu des messages sont chiffres avec AES-256-CBC. La cle derive d'un secret via SHA-256, comme dans l'implementation Node, pour permettre l'interopabilite des documents existants.



**Figure 2.7 — AES-256-CBC Encryption Pipeline**



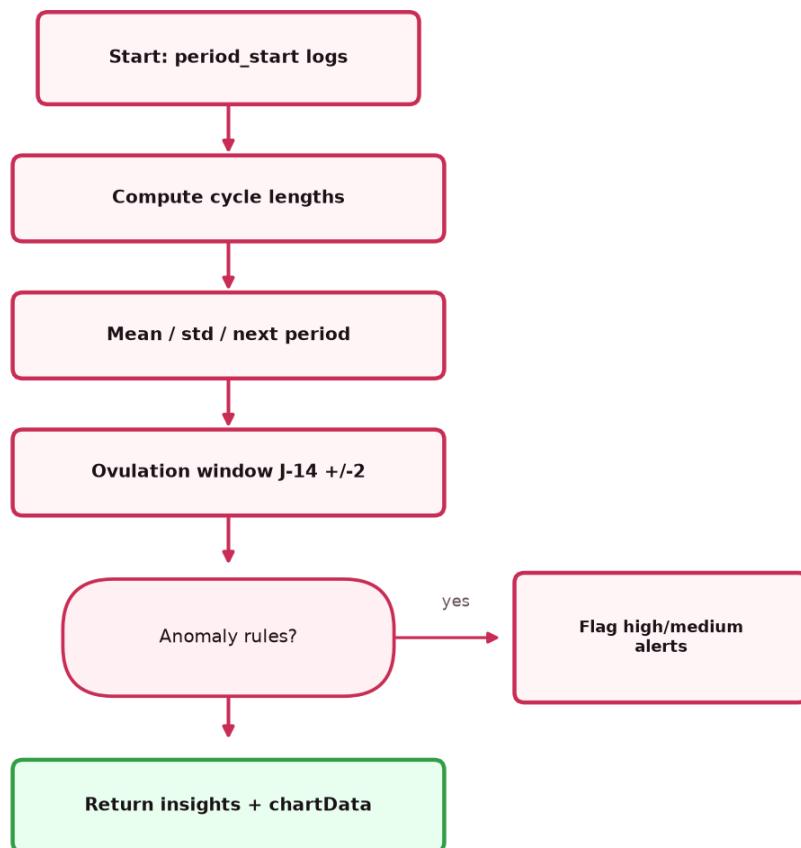
*Decrypt only on server after JWT + RBAC authorization*

*Figure: chiffrement des donnees de sante*

### 4.3 Analyse predictive

Le service HealthScoreService calcule un score 0-100 a partir du sommeil, stress, activite, humeur et glycemie. CycleAnalyzer estime phase, ovulation et anomalies.

**Figure 2.8 — Predictive Cycle Analysis Flowchart**



*Figure: pipeline predictif*

## 5. Realisation backend .NET

### 5.1 Authentication

AuthController expose register/login/me/doctors/assign-doctor. Les mots de passe sont hashés avec BCrypt (work factor 12). JwtTokenService émet un token HMAC-SHA256 valide 7 jours. Les controllers utilisent [Authorize(Roles=...)].

### 5.2 Modules métier

- HealthController: symptomes, periodes, insights, dossier chiffre
- SuiviController: daily upsert, tendance, diabete
- ChatController: partenaires + messages chiffres
- AiController: chat, coach-plan, wellness, doctor-brief
- AdminController: users, plans, subscriptions, audit

### 5.3 Enhancements

- Appointments: reservation video/chat/presentiel + notifications
- Medications: rappels horaires pour adherence therapeutique
- Notifications: centre d'alertes in-app
- AuditLog: trace des actions admin
- Swagger UI: documentation interactive /swagger

**Figure 3.1 — Backend Folder Structure**

```
backend/  
├─ api/index.js           (Vercel serverless entry)  
├─ src/app.js             (Express app)  
├─ src/server.js          (local listen + Socket.io)  
├─ src/config/db.js  
├─ src/models/            User, HealthRecord, SymptomLog, Message  
├─ src/controllers/       auth, health, chat, ai  
├─ src/routes/  
├─ src/middlewares/auth.middleware.js  
├─ src/utils/             crypto.js, analyzer.js, claude.js  
└─ src/sockets/chat.socket.js
```

Figure: organisation backend

**Figure 3.2 — Frontend Route Map**

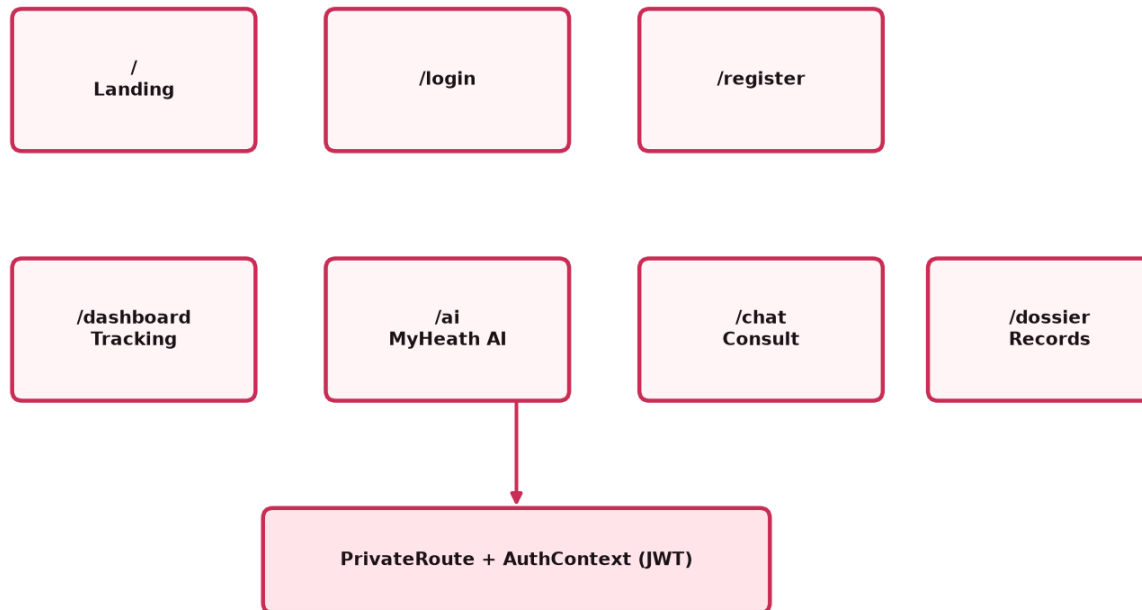


Figure: cartographie des routes API

## 6. Frontend React

Le frontend conserve Vite/React/Tailwind avec un layout dashboard (sidebar). Les pages Appointments et Medications consomment les nouvelles routes .NET. La variable VITE\_API\_URL selectionne le backend (Node ou .NET).

- Admin console: Overview, Users, Subscriptions, Plans
- Patient: Suivi, Period, Diabetes, Medications, AI Coach, Records
- Commun: Appointments, Consult (chat)

## 7. Docker et deployment

### 7.1 Conteneurisation

Le Dockerfile multi-stage utilise le SDK .NET 8 pour publier l'API, puis l'image runtime aspnet:8.0 pour executer MyHeath.Api.dll sur le port 5080. docker-compose.dotnet.yml orchestre mongodb + backend-dotnet + frontend.

```
docker compose -f docker-compose.dotnet.yml up --build
# API http://localhost:5080/api/healthcheck
# Swagger http://localhost:5080/swagger
# UI http://localhost:5173
```

### 7.2 Variables d'environnement

- MONGODB\_URI / MONGODB\_DATABASE
- JWT\_SECRET / AES\_SECRET\_KEY
- CLIENT\_URL (CORS)
- ANTHROPIC\_API\_KEY / ANTHROPIC\_MODEL
- PORT=5080

### 7.3 Deployment cloud

Frontend: Vercel (build Vite, VITE\_API\_URL vers l'API publique). API .NET: Azure App Service, Railway, Render (Docker), ou VM avec Docker Compose. MongoDB: Atlas (URI srv) en production. Les secrets ne doivent jamais etre commits.

**Figure 2.2 — Deployment Topology**

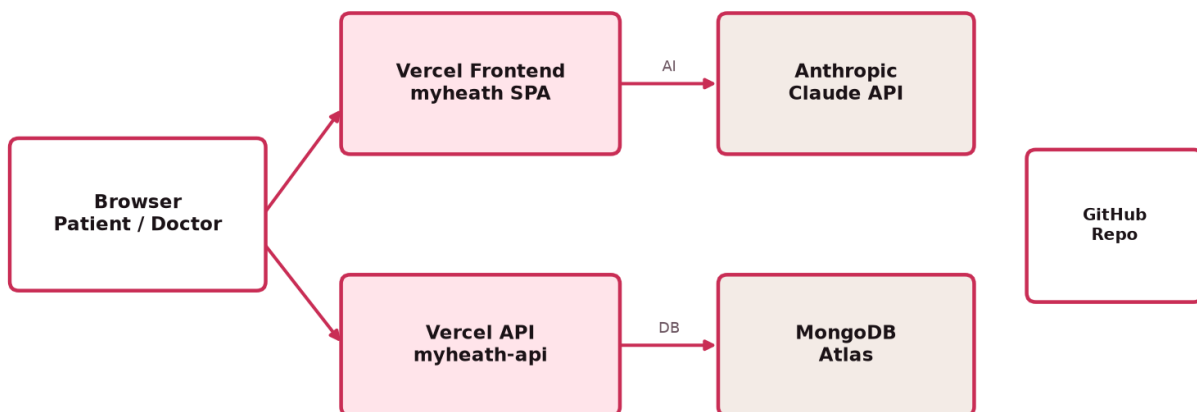


Figure: vue deployment

### 7.4 Pipeline recommande

- CI: build docker image + tests smoke /api/healthcheck
- CD: push image registry -> deploy App Service / Compose

- Frontend: vercel --prod apres mise a jour VITE\_API\_URL

## 8. Tests et comptes de demonstration

Au demarrage, SeedService cree les comptes et plans si absents.

- admin@myheath.app / Admin123
- doctor@myheath.app / Doctor123
- patient@myheath.app / Patient123 (Care)
- man@myheath.app / Patient123 (Premium)

### 8.1 Scenarios de validation

- Login admin -> console -> assigner un abonnement
- Patient -> suivi quotidien -> score
- Patient -> book appointment -> notification medecin
- Patient -> medication reminder
- AI Coach (si ANTHROPIC\_API\_KEY presente)
- Chat chiffre patient-medecin

## 9. Comparaison Node.js vs ASP.NET Core

Les deux backends exposent le meme contrat API. Node (Express) reste utile pour Socket.io temps reel local; .NET apporte typage fort, DI native, Swagger, et une base solide pour extensions entreprise (policies, health checks, Azure).

- Node: prototypage rapide, Vercel serverless deja en place
- .NET: architecture layers, performances, competences academiques C#



## 10. Conclusion et perspectives

Ce PFE demontre la migration d'une API telemedecine vers ASP.NET Core 8 tout en preservant React et MongoDB, avec Docker et une strategie de deploiement claire. Perspectives: SignalR pour le chat temps reel, paiements abonnements, app mobile, et audits de conformite Loi 09-08 / RGPD enrichis.

### Livrables

- Code: backend-dotnet/, frontend/, docker-compose.dotnet.yml
- Rapport: docs/RAPPORT\_PFE\_MYHEATH\_DOTNET.pdf
- API docs: /swagger

## 11. Securite approfondie

### 11.1 Surface d'attaque

Les principaux risques concernent l'usurpation de session JWT, l'accès croisé aux dossiers patients, et l'exposition de secrets (clés AES, Anthropic). Les contrôleurs vérifient systématiquement le rôle et l'appartenance des ressources.

### 11.2 Contre-mesures

- HTTPS obligatoire en production (TLS termine par reverse proxy / Azure)
- Secrets injectés via variables d'environnement, jamais dans Git
- CORS restreint à CLIENT\_URL + domaine Vercel
- Validation des DTO et longueurs maximales sur les prompts IA
- Désactivation douce des utilisateurs (isActive) plutôt que suppression dure
- HTTPS obligatoire en production (TLS termine par reverse proxy / Azure)
- Secrets injectés via variables d'environnement, jamais dans Git
- CORS restreint à CLIENT\_URL + domaine Vercel
- Validation des DTO et longueurs maximales sur les prompts IA
- Désactivation douce des utilisateurs (isActive) plutôt que suppression dure

### 11.3 Conformité

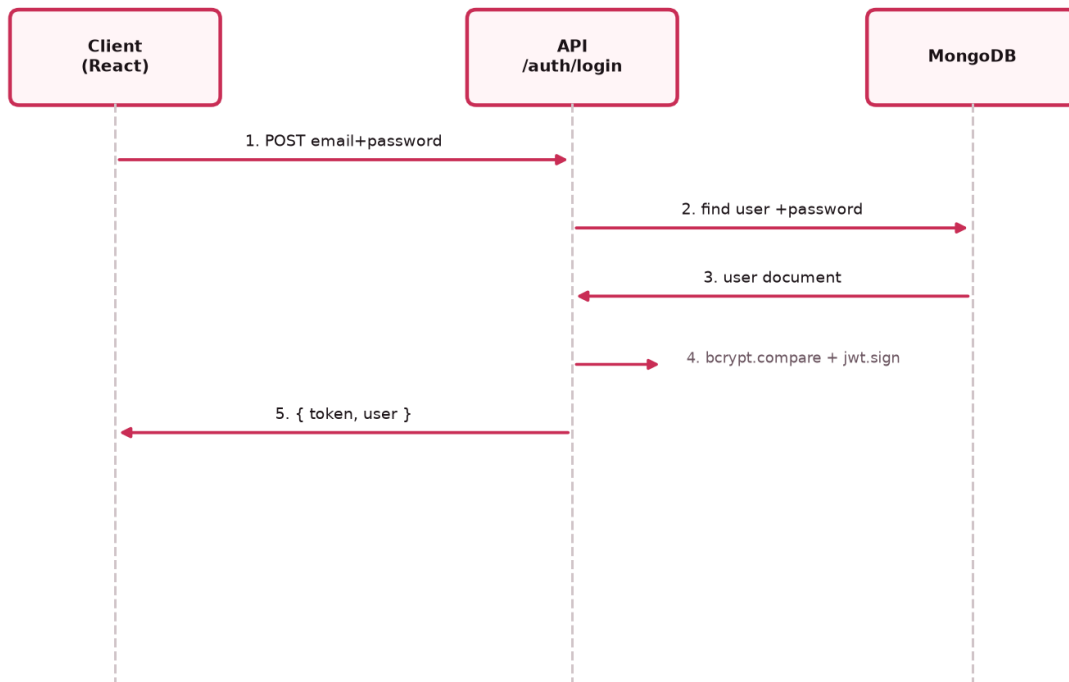
Le chiffrement au repos des notes cliniques et messages s'aligne sur les bonnes pratiques Loi 09-08 (Maroc) et les principes RGPD (minimisation, intégrité, confidentialité). L'IA affiche un disclaimer non diagnostique.

## 12. Scenarios UML (narratifs)

### 12.1 Sequence login

Le client POST /api/auth/login -> AuthController verifie BCrypt -> JwtTokenService emet le token -> le frontend stocke myheath\_token et appelle /api/auth/me.

**Figure 2.4 — Sequence: Login & JWT**



*Figure: sequence d'authentification*

### 12.2 Sequence insights

Le patient GET /api/health/insights -> chargement SymptomLog -> CycleAnalyzer -> reponse JSON (phase, ovulation, anomalies).

**Figure 2.5 — Sequence: Insights Computation**

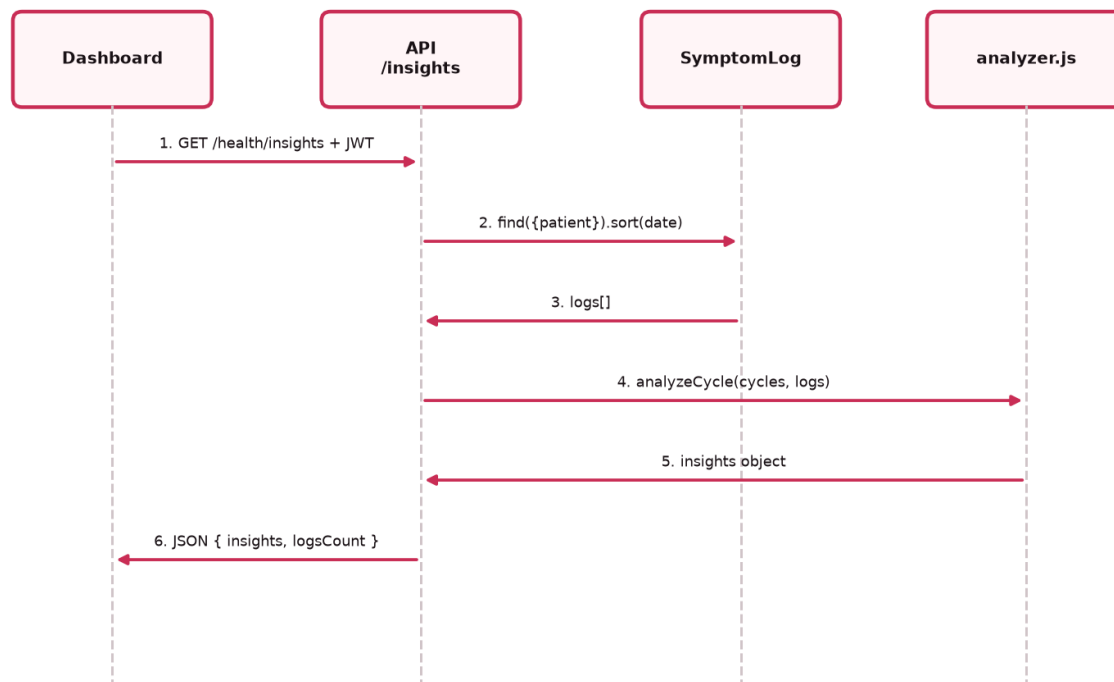


Figure: sequence calcul d'insights

## 12.3 Sequence AI Coach

Le patient POST /api/ai/coach-plan -> contexte DailyHealthLog 7j -> ClaudeService appelle Anthropic -> plan retourne avec disclaimer.

**Figure 2.9 — MyHeath AI Contextual Prompt Pipeline**

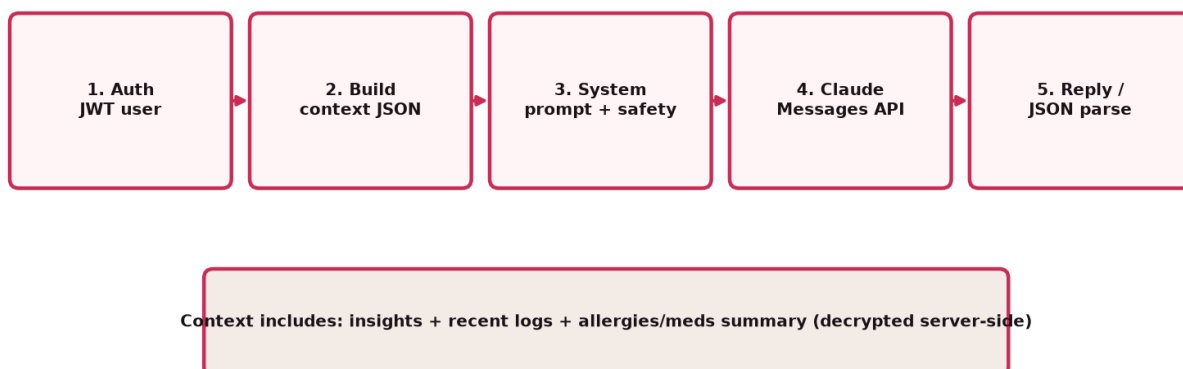


Figure: integration IA Claude

## 13. Guide operateur Docker (detail)

### 13.1 Prerequis

- Docker Desktop 4.x ou Docker Engine + Compose v2
- 8 Go RAM recommandes pour build SDK .NET
- Ports libres: 27017, 5080, 5173

### 13.2 Commandes utiles

```
docker compose -f docker-compose.dotnet.yml build --no-cache
docker compose -f docker-compose.dotnet.yml up -d
docker compose -f docker-compose.dotnet.yml logs -f backend-dotnet
docker compose -f docker-compose.dotnet.yml down -v
```

### 13.3 Depannage

- Cas 1: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 2: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 3: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 4: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 5: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 6: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.
- Cas 7: si healthcheck echoue, tester curl `http://localhost:5080/api/healthcheck` et inspecter MONGODB\_URI / firewall.

Le volume `mongo_data_dotnet` persiste les documents entre redemarrages. Pour reinitialiser le seed, supprimer le volume (-v) puis relancer.

## 14. Deploiement Azure (scenario)

Scenario recommande pour une soutenance professionnelle: Azure Container Registry + Azure App Service (Linux container) pour l'API, MongoDB Atlas M0/M10, frontend Vercel. Les App Settings Azure mappent les memes variables que Compose (JWT\_SECRET, AES\_SECRET\_KEY, MONGODB\_URI, ANTHROPIC\_API\_KEY).

- Etape Azure 1: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 2: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 3: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 4: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 5: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 6: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 7: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 8: configurer App Setting puis redemarrer le conteneur API
- Etape Azure 9: configurer App Setting puis redemarrer le conteneur API

Le healthcheck /api/healthcheck peut etre branche sur Azure Health Check path pour le recyclage automatique en cas d'echec.

## 15. Plan de tests

### 15.1 Tests fonctionnels

- Parcours Admin: login, navigation sidebar, actions metier, logout
- Parcours Medecin: login, navigation sidebar, actions metier, logout
- Parcours Patient femme: login, navigation sidebar, actions metier, logout
- Parcours Patient homme: login, navigation sidebar, actions metier, logout

### 15.2 Tests API (Swagger)

Swagger permet d'executer les endpoints avec Authorize Bearer. Verifier les codes 401/403 sur les routes protegees.

- Smoke: GET /api/healthcheck
- Smoke: POST /api/auth/login
- Smoke: GET /api/admin/overview
- Smoke: POST /api/suivi/daily
- Smoke: GET /api/appointments
- Smoke: POST /api/medications
- Smoke: GET /api/notifications
- Smoke: POST /api/ai/chat

### 15.3 Criteres d'acceptation

- Aucun secret dans le depot Git
- Compose up demarre les 3 services sans erreur fatale
- Frontend affiche dashboard apres login
- Messages et dossier restent lisibles apres chiffrement/dechiffrement

## Annexe A - Exemples de requetes

POST /api/auth/login

```
{ "email": "patient@myheath.app", "password": "Patient123" }
```

POST /api/suivi/daily

```
{ "sleepHours": 7.5, "energy": 7, "stress": 3, "mood": "good" }
```

POST /api/appointments

```
{ "doctorId": "...", "scheduledAt": "2026-08-01T10:00:00Z", "mode": "video" }
```



## Annexe B - Checklist Docker

- Checklist 1: verifier logs du service backend-dotnet apres compose up
- Checklist 2: verifier logs du service backend-dotnet apres compose up
- Checklist 3: verifier logs du service backend-dotnet apres compose up
- Checklist 4: verifier logs du service backend-dotnet apres compose up
- Checklist 5: verifier logs du service backend-dotnet apres compose up
- Checklist 6: verifier logs du service backend-dotnet apres compose up
- Checklist 7: verifier logs du service backend-dotnet apres compose up
- Checklist 8: verifier logs du service backend-dotnet apres compose up
- Checklist 9: verifier logs du service backend-dotnet apres compose up
- Checklist 10: verifier logs du service backend-dotnet apres compose up
- Checklist 11: verifier logs du service backend-dotnet apres compose up
- Checklist 12: verifier logs du service backend-dotnet apres compose up
- Checklist 13: verifier logs du service backend-dotnet apres compose up
- Checklist 14: verifier logs du service backend-dotnet apres compose up
- Checklist 15: verifier logs du service backend-dotnet apres compose up

En cas d'echec de connexion Mongo, verifier le hostname mongodb dans le reseau Compose et l'URI MONGODB\_URI=mongodb://mongodb:27017.

## Annexe C - Glossaire

- JWT: JSON Web Token pour sessions API
- AES: Advanced Encryption Standard
- RBAC: Role-Based Access Control
- MRR: Monthly Recurring Revenue (abonnements)
- DI: Dependency Injection (.NET)
- Swagger: Documentation OpenAPI interactive
- Atlas: MongoDB Database-as-a-Service
- Vercel: Hébergement frontend JAMstack
- Compose: Orchestration multi-conteneurs Docker
- SignalR: Perspective temps reel .NET (evolution chat)

## Annexe D - Bibliographie / references techniques

- Microsoft Learn - ASP.NET Core 8 documentation

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- MongoDB C# Driver documentation

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- OWASP ASVS - authentication & cryptography chapters

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- Anthropic API - Messages endpoint

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- Docker multi-stage builds best practices

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- Loi 09-08 relative a la protection des personnes physiques (Maroc)

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.

- RGPD - principes de minimisation et securite des traitements

Reference utilisee pour justifier les choix d'architecture et de securite du PFE MyHeath .NET.